DevOps & Tools · revision notes

Git & GitHub

Commits, branches, merge vs rebase, undoing, team workflow

1 How Git thinks

Definition

Git is a distributed version control system. A commit is a snapshot of the project + a pointer to its parent(s); a branch is just a movable pointer to a commit.

Think of Git like saving a video game

1
Playing
Working tree
You change files as you play

2
Choose what to save
Staging area
git add picks the changes

3
Save slot
Commit
A snapshot you can always return to

4
Cloud save
Remote (GitHub)
git push shares it with the team

Working tree → git add → Staging area → git commit → Local repo → git push → Remote (GitHub)

Everyday commands

```
git status                      # what changed?
git add -p                      # stage chunks interactively
git commit -m "Add login form validation"
git switch -c feature/login     # new branch
git pull --rebase               # update without merge bubbles
git push -u origin feature/login
git log --oneline --graph --all
git diff / git diff --staged
```

2 Merge vs rebase

Think of merge vs rebase like combining two story drafts

1



Two writers branch off

Branches

Each adds their own chapters

2



Staple the drafts together

git merge

Keeps both histories, adds a merge commit

3



Rewrite as if one after another

git rebase

Moves your commits on top: straight line

4



Do not rewrite shared drafts

Golden rule

Never rebase a branch others use

	merge	*rebase*
History	keeps the true history + a merge commit	rewrites commits on top of the target → linear
Safe on shared branches?	yes	no — never rebase commits others have pulled
Conflicts	resolved once	possibly per commit

- Squash merge: the whole PR becomes one commit on main (clean history).
- Resolve conflicts: edit the marked files, `git add`, then `git merge --continue` / `git rebase --continue`.
- Golden rule: rebase your own local work; merge shared branches.

3 Undoing things

Think of undoing like the undo options in a document

1



Throw away unsaved edits

`git restore`

Discard changes in a file

2



Take it off the shelf

`git reset`

Unstage or move the branch back

3



Add a "reverse" page

`git revert`

Safe: new commit that cancels the old one

4



Time machine

`git reflog`

Find and recover lost commits

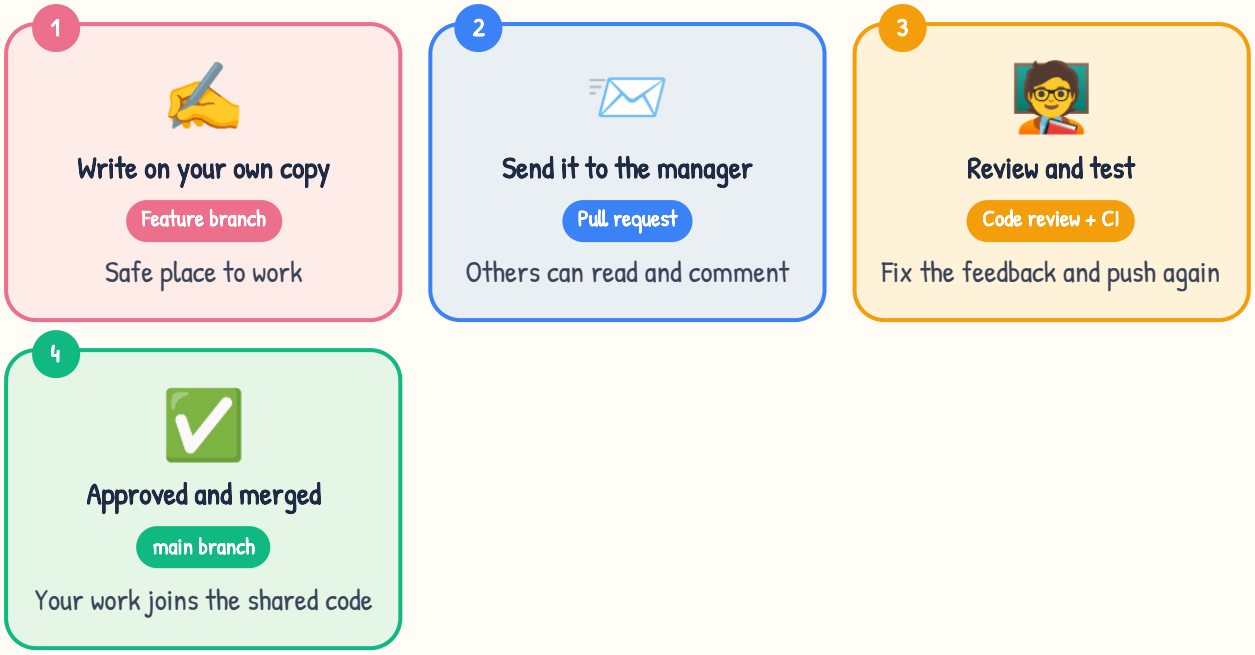
Situation	Command
Discard changes to a file	`git restore file`
Unstage a file	`git restore --staged file`
Fix the last commit message / add a file	`git commit --amend` (only if not pushed)
Undo a pushed commit safely	`git revert <sha>` (new commit that reverses it)
Move the branch back, keep changes	`git reset --soft HEAD~1`
Throw local commits away	`git reset --hard <sha>` (destructive)
Find "lost" commits	`git reflog`
Park work in progress	`git stash` / `git stash pop`
Copy one commit to another branch	`git cherry-pick <sha>`
Find the commit that broke something	`git bisect`

Common mistake

`git push --force` on a shared branch overwrites teammates' work. Use `--force-with-lease`, and only on your own branches.

4 Team workflow on GitHub

Think of a pull request like submitting a report for review



1. Create a short-lived feature branch from `main`.
2. Commit small, focused changes with clear messages (Conventional Commits: `feat:`, `fix:`, `chore:`).
3. Open a pull request; CI runs lint, tests and build.
4. Review, address comments, keep the branch updated.
5. Squash-merge, delete the branch, deploy.

- Protect `main`: required reviews and passing checks.
- `.gitignore` build output, `node_modules`, `.env`.
- Tags (`v1.4.0`) mark releases; semantic versioning.
- Never commit secrets — if you do, rotate them (history keeps them).

Revision

Commit = snapshot, branch = pointer. Merge shared branches, rebase local ones. revert for pushed commits, reset for local ones, reflog to recover. Small PRs, protected main, CI on every PR.

5 Rapid-fire Q&A

Q. git merge vs git rebase?

A. Merge joins histories with a merge commit and keeps them as they were; rebase replays your commits on top of another branch for a linear history but rewrites commit ids.

Q. git reset vs git revert?

A. reset moves the branch pointer (rewrites history, for local work); revert adds a new commit that undoes an earlier one (safe for pushed commits).

Q. git fetch vs git pull?

A. fetch downloads remote changes without touching your branch; pull = fetch + merge (or rebase).

Q. What is the staging area?

A. The index where you prepare exactly what goes into the next commit.

Q. How do you recover a commit after a bad reset?

A. Find its hash in git reflog and reset or cherry-pick it back.

Q. What does git stash do?

A. Saves uncommitted changes and cleans the working tree so you can switch tasks, then restores them with stash pop.

Q. What is cherry-pick?

A. Applying the changes of one specific commit onto the current branch.

Q. How do you resolve a merge conflict?

A. Edit the conflicted files to the right content, remove the markers, git add them, then continue the merge or rebase.

Q. Why is force-pushing dangerous?

A. It overwrites the remote history, deleting commits others may have pulled; use --force-with-lease and only on your own branches.